

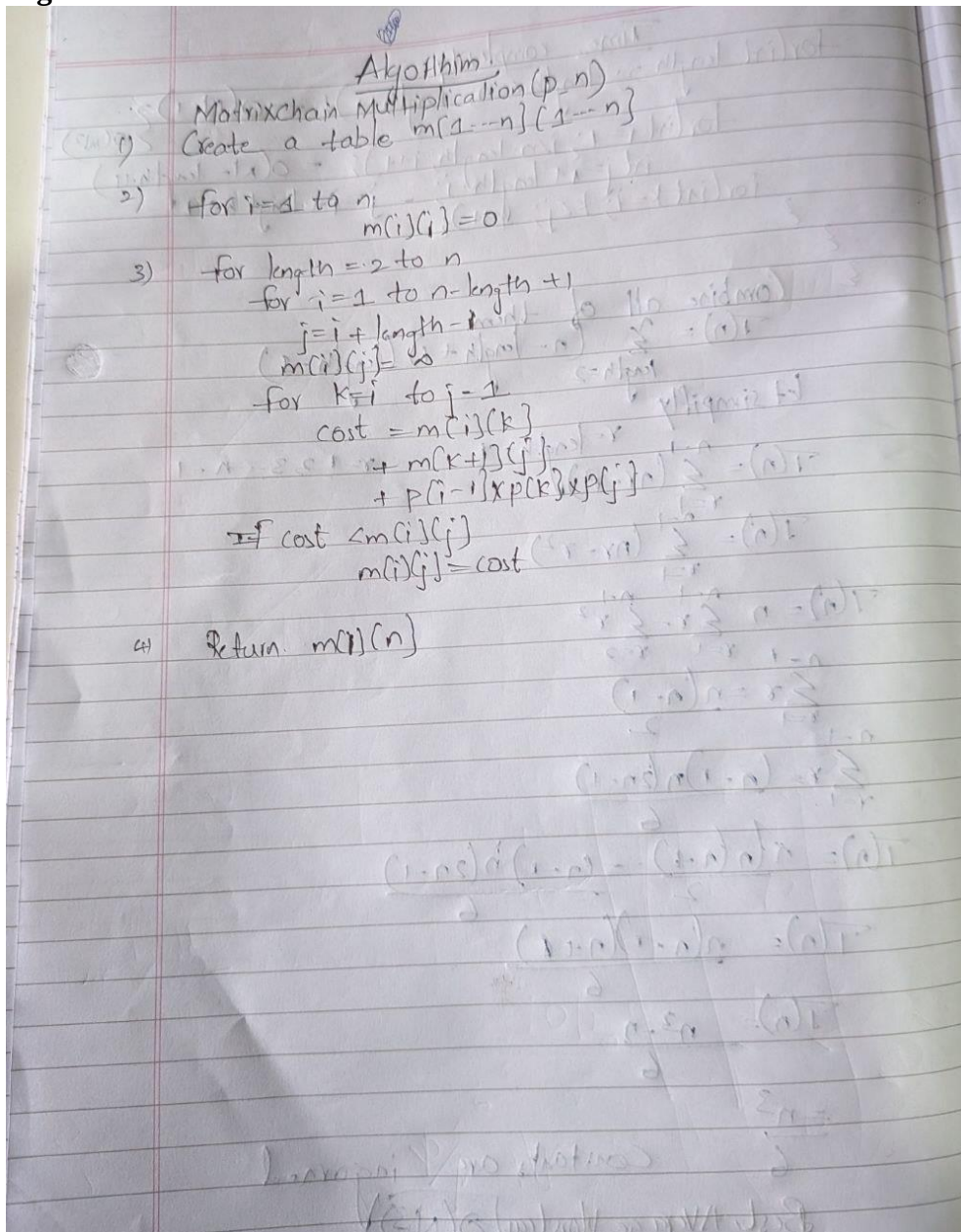
Practical: 6

Implementation of chain matrix multiplication using dynamic programming

01/09/2026

Aim: To Implement the chain matrix multiplication using dynamic programming

Algorithm:



Program:

```
#include <iostream>
using namespace std;

int main(){
    int n = 5;

    int p[] = {5, 10, 3, 12, 5, 50};

    int c[10][10] = {0};
    int k[10][10] = {0};

    for(int d = 1; d < n; d++){
        for(int i = 1; i <= n-d; i++){
            int j = i + d;
            int min = 99999;

            for(int x = i; x < j; x++){
                int q = c[i][x] + c[x+1][j]
                    + p[i-1] * p[x] * p[j];

                if(q < min){
                    min = q;
                    k[i][j] = x;
                }
            }
            c[i][j] = min;
        }
    }

    cout << "\nMinimum cost = " << c[1][n]<<endl;

    return 0;
}
```

Output:

```
Minimum cost = 1655
Optimal split positions:
k[1][2] = 1
k[1][3] = 2
k[1][4] = 2
k[1][5] = 4
k[2][3] = 2
k[2][4] = 2
k[2][5] = 2
k[3][4] = 3
k[3][5] = 4
k[4][5] = 4
...Program finished with exit code 0
Press ENTER to exit console.
```

Time Complexity:

Time complexity:

$$\text{for(int length=2; length<=n; length++)} \quad \{ \quad \} \quad \sim O(N-1) \quad \{ \quad \} \quad \sim O(N^2)$$

$$\text{for(int i=1; i<=length; i++)} \quad \{ \quad \} \quad \sim O(N-1) \quad \{ \quad \} \quad \sim O(N^2)$$

$$\text{int j=i+length-i; } \quad \text{for(int k=i; k<=j; k++)} \quad \{ \quad \} \quad \sim O(N-1) \quad \{ \quad \} \quad \sim O(N^2)$$

Combine all of them.

$$T(n) = \sum_{\text{length}=2}^{n-1} (n - \text{length} + 1) (\text{length} - 1)$$

Let simplify

$$T(n) = \sum_{r=1}^{n-1} (nr - r^2) \quad r = \text{length} - 1 \quad r = 1, 2, 3, \dots, n-1$$

$$T(n) = \sum_{r=1}^{n-1} (nr - r^2)$$

$$T(n) = n \sum_{r=1}^{n-1} r - \sum_{r=1}^{n-1} r^2$$

$$\sum_{r=1}^{n-1} r = \frac{n(n-1)}{2}$$

$$\sum_{r=1}^{n-1} r^2 = \frac{(n-1)n(2n-1)}{6}$$

$$T(n) = \frac{n(n-1)}{2} - \frac{(n-1)n(2n-1)}{6}$$

$$T(n) = \frac{n(n-1)(n+1)}{6}$$

$$T(n) = \frac{n^3 - n}{6}$$

$$\frac{n^3}{6}$$

Constants are ignored

Best, Average, Worst case $\boxed{O(N^3)}$

Conclusion:

The Implementation of the of chain matrix multiplication using dynamic programming has successfully executed with the time complexity of $O(n^3)$ in the Three cases of best, average and worst. At the end of this program says that the Minimum number of scalar multiplications required to multiply all matrices.